Nama : Arief Purnama Muharram
NIM : 23521013

SQL → Stored in columns and tables

DML → Data Manipulation Language → Insert, update, delete, select

DDL → Data Definition Language
- → Primary key
- → Composite key
- → Foreign key

ACID

A : Atomic → transaction complete or not

C : Consistent → transaction is consistent
→ defind in terms of constraint

I : Isolated → the result of any changes made during a transaction are not visible until the transaction has committed

D : Durable → The result of a committed transaction survive failures

BASE → in NoSQL

Basically Available → Nodes can go up and down but the system shouldn't affected

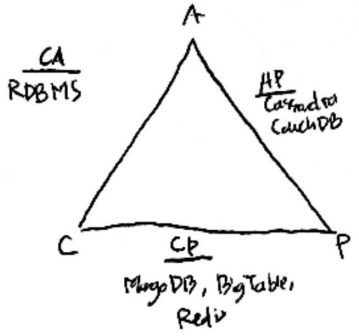
Soft State (Scalable) → The state of the system and data changes over time

Eventually Consistent → Given enough time, data will be consistent across the distributed system ("pada akhirnya sinkron juga")

Brewer's CAP Theorem

A distributed system can only support two of following characteristics;

Consistency ⇒ All nodes see the same data at the same time
Availability → Node failures doesn't make the rest to be failed
Partition tolerance ⇒ The system continues to operate despite arbitrary message loss



NoSQL Database Types

Key-Value Store
Column Store : Hadoop, Cassandra
Document Store
Graph model

Key-Values

Pros : Fast, Scalable, Simple, easy to distribute horizontally

Cons : Many data cannot be modeled

Key-Value DB

Key ≠ referred

Key convention:
<table> : <primary-key> : <attribute>

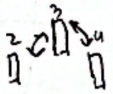
Namespace → Collection of ~~key pairs~~ key-value pairs
→ define an additional profit for the keys.

Cluster

→ Sets to connected computers that coordinate their operations
→ key value → loosely coupled
→ type → Masterless (ex Riak)
→ Have master (ex Redis)

Ring

→ Circular pattern in which each server or instance of key-value database software running on a server is linked to two adjacent servers or instances.



Replication

→ process of saving multiple copies of data in a cluster

Collision

→ occurs when two distinct input to a hash function produce the same output.
→ Compression technique.

Design Pattern for Key-Value Databases

Time-to-Live (TTL) keys
Embedding tables
Aggregates
Atomic aggregates
Enumerable keys
Indexes

---

① Document ⇒ # set of ordered key-value pairs

Collection → A group of documents

Embedded document → enables document database users to store related data in a single document.

Polymorphic → many forms

Type of partitions : # Vertical partitioning
Horizontal partitioning

Vertical → column splitting

Horizontal → row splitting

split by shard key

Tuning : 1. Index tuning
2. Schema tuning
3. Query tuning
4. Transaction tuning

Index methods : # Ordered index
B^+-tree index
Hash index
Bitmap index

>> create index <index-name> on <rel-name> (<attribute-list>)

>> create index b-index on branch(branch-name)

>> drop index <index-name>

Schema tuning → Horizontal decomposition
→ Vertical composition

Splitting
Horizontally
Vertically
⇓
column-based

Denormalization
Adding redundant columns
Adding derived attributes
Collapsing tables
Duplicate tables
→ menambah kolom redundan
→ menduplikasi kolom derived atribut

Ensure data integrity :
Trigger
Application logic
Batch reconciliation

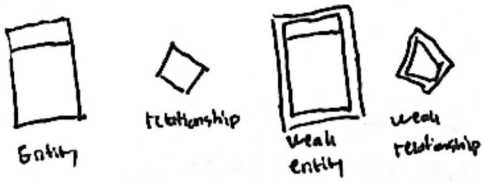
Materialized View ⇒ Simpan tabel
>> CREATE MATERIALIZED VIEW <name> AS
>> SELECT ...

Normalisasi bertujuan untuk:
menghindari insertion anomaly
menghindari deletion anomaly
menghindari modification anomaly

Transitive dependency ⇒ when some non-key attribute determines some other attribute

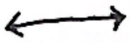
Scanned with CamScanner

Nama : Arief Purnama Muhaimin
Nim : 23521013



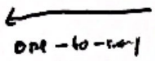
Relationship Sets w/ Attr

Relationships

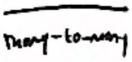


one-to-one

many-to-one



one-to-many



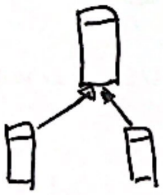
many-to-many

Total participation

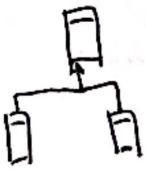
Many-to-one
One-to-many } lebar ke sisi many

many yang punya primary key

kalau one-to-one, pilih salah satu saja!



Overlapping
(bisa juga bisa)



Disjoint
(salah satu aja)

>> CREATE TABLE <name> (
<attribute> <type> PRIMARY KEY
FOREIGN KEY ON (<attr>) REFERENCES <table>(<attr>)
)

Data type

CHAR (size)
VARCHAR (size)
BINARY (size)
TEXT (size)
BOOLEAN
INT (size)
BIGINT (size)
FLOAT (size, d)
DOUBLE (size, d)
DATETIME
TIMESTAMP
TIME
YEAR

>> DROP TABLE <name>

>> TRUNCATE TABLE <name>

>> ALTER TABLE <name> ADD <col> <type>

>> INSERT INTO ... (...) VALUES (...)

>> UPDATE <name> SET <attribute> = "..."

>> PRIMARY KEY(..., ..., ...)

>> SELECT EXTRACT(YEAR, <date>) AS <attr>

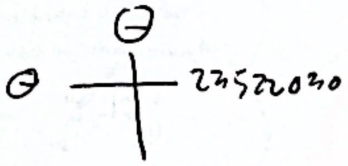
>> DELETE FROM <table-name> WHERE <condition>

>> AVG(...)

>> COUNT(...)

>> CREATE TRIGGER <name> {BEFORE | AFTER}
{INSERT | UPDATE | DELETE} ON <table> FOR
EACH ROW BEGIN ... END$$ DELIMITER ;

(4)



23322050

Scanned with CamScanner